Unit III

3. Tools of structured Analysis

- **3.1 Data Flow diagram**
- **3.2 Data dictionary**
- **3.3 IPO charts**
- **3.4 HIPO charts**
- **3.5 Gantt charts**
- **3.6 Pseudo codes**
- **3.7 Flow charts**
- **3.8 Decision tree**
- **3.9 Decision tables**

3.1 Data Flow diagram:

Definition:

A **Data Flow Diagram (DFD)** is a graphical tool used to represent the **flow of data** in a system. It shows **how data moves** between processes, data stores, and external entities.

Purpose of DFD:

To **visually model** how data is processed in a system.
Helps analysts and developers understand the system's **inputs, processes, and outputs**.

Key Components of a DFD:

1. Processes (Circles or Ovals):
Show how data is transformed or processed.

2. Data Flows (Arrows):
Indicate the direction and movement of data between elements.

3. Data Stores (Open-ended Rectangles):
Represent places where data is stored for later use.

4. External Entities (Squares or Rectangles):
Represent outside sources or destinations of data (e.g., users, other systems).

Types of DFDs:

Level 0 (Context Diagram): Shows the entire system as a single process.
Level 1, 2, etc.: Break down the system into more detailed sub-processes.

Conclusion:

DFDs are useful for understanding and analyzing system requirements and designing efficient data processing systems.

3.2 Data dictionary:

Definition:

A **Data Dictionary** is a **centralized repository** that stores **definitions and details** about all data elements used in a system. It acts like a reference guide for developers, analysts, and users.

Purpose:

To provide **clear and consistent descriptions** of data elements.
To help in **system design, development, and maintenance**.
Ensures **standardization** of data across the system.

Contents of a Data Dictionary:

1. **Data Name –** The name of the data element.
2. **Description –** What the data represents.
3. **Type –** The data type (e.g., Integer, String, Date).
4. **Size/Length –** Maximum number of characters or digits.
5. **Allowed Values –** Valid range or options (e.g., Yes/No).
6. **Source –** Where the data comes from.
7. **Destination –** Where the data is used or stored.

Example Entry:

Data Name	Type	Size	Description	Allowed Values
Student_ID	String	10	Unique ID of student	Alphanumeric
Issue_Date	Date	------	Date of book issue	Valid date

Conclusion:

The data dictionary is essential for **avoiding confusion**, maintaining **data integrity**, and improving **system documentation**.

3.3 IPO charts:

IPO Charts (Input-Process-Output Charts):

Definition:

An **IPO Chart** is a simple tool used to represent the **Input**, **Process**, and **Output** of a system or a function. It helps in understanding how a task is performed step-by-step.

Purpose:

To **break down** a system or function into basic components.
Helps analysts and developers **design and document** systems clearly.
Useful in both **programming** and **system analysis**.

Structure of an IPO Chart:

Input	Process	Output
Data entered into system	Task or operation performed	Result or outcome produced

Example: Payroll System IPO Chart:

Input	Process	Output
Employee ID, Hours Worked, Pay Rate	Calculate salary (Hours x Rate), Apply deductions	Net Salary Slip

Conclusion:

IPO charts help in **organizing logic** clearly, making system design and documentation more

effective and understandable.

3.4 HIPO charts:

Definition:

HIPO (Hierarchy plus Input-Process-Output) charts are a **visual tool** used in system analysis and design to show a system's **structure (hierarchy)** and the **details of each function** in terms of input, process, and output.

Purpose:

To provide a **top-down view** of the system.
Breaks down the system into **modules** and **sub-modules**.
Helps in better **understanding, designing, and documenting** large systems.

Components of HIPO Charts:

1. Hierarchy Chart:

Shows the **overall structure** of the system.
Main modules at the top, sub-modules underneath.

2. IPO Charts for Each Module:

Describes **what each module does** using:

Input – What data goes in
Process – What the module does
Output – What result comes out

Example (Library System):

Hierarchy Chart:

```
Library System
    ├── Issue Book Module
    ├── Return Book Module
    └── Manage Members Module
```

IPO for "Issue Book Module":

Input	Process	Output
Book ID, Member ID	Check availability, update record	Confirmation message

Conclusion:

HIPO charts help in **breaking down complex systems**, making them easier to **design, understand, and develop**.

3.5 Gantt charts:

Definition:

A **Gantt Chart** is a type of **bar chart** used in **project management** to visually represent a project's schedule. It shows **tasks or activities** along with their **start and end dates**, making it easy to track progress over time.

Purpose:

Helps in **planning, scheduling, and monitoring** project activities.
Useful for **system development**, showing each phase of the SDLC (System Development Life Cycle).
Helps in **resource allocation** and identifying **task dependencies**.

Key Features:

1. **Tasks/Activities:** Listed on the vertical (Y) axis.
2. **Time Scale:** Shown on the horizontal (X) axis (days, weeks, etc.).
3. **Bars:** Horizontal bars show the **duration** of each task.
4. **Dependencies:** May show how one task depends on another.

Example (Mini Project Gantt Chart):

Task	Start	End	Duration
Requirement Analysis	Day 1	Day 3	3 days
System Design	Day 4	Day 6	3 days
Development	Day 7	Day 12	6 days

- Bars drawn across the timeline show when each task happens.

Conclusion:

Gantt charts provide a **clear visual timeline**, helping project managers keep the system project **on track** and well-organized.

3.6 Pseudo codes:

Definition:

Pseudocode is a **simplified, informal way of writing a program's logic** using plain English-like statements. It does not follow any specific programming language syntax but helps in planning and designing code.

Purpose:

To **outline the logic** of a program before actual coding.
Helps programmers and analysts **focus on the steps** rather than syntax.
Makes it easier to **translate logic into any programming language** later.

Key Features:

1. **Simple and readable** language.
2. **Structured like real code** with loops, conditions, and functions.
3. No strict syntax – focus is on logic, not formatting.

Example:

Task: Find the sum of two numbers

```
START
    INPUT number1, number2
    sum ← number1 + number2
    PRINT sum
END
```

Conclusion:

Pseudocode is a valuable tool in **system analysis and programming**. It simplifies complex logic,

improves communication among team members, and makes the coding process smoother and error-free.

3.7 Flow charts:

Definition:

A **flowchart** is a **diagrammatic representation** of an algorithm or process, using **symbols** to show the **flow of control** and **steps** involved in solving a problem or performing a task.

Purpose:

To visually describe **how a process or system works**.
Helps in **understanding, analyzing, and debugging** the logic before implementation.
Useful in **system design, programming, and documentation**.

Common Flowchart Symbols:

Symbol	Meaning
Rectangle	**Process/Operation** step
Diamond	**Decision/Condition** (Yes/No)
Oval	**Start/End** of the flow
Arrow	**Flow direction between steps**

Conclusion:

Flowcharts are an essential tool for **planning and explaining logic** clearly. They are easy to understand and help reduce errors during system or program development.

3.8 Decision tree:

Definition:

A **decision tree** is a **graphical tool** used to represent **decision-making logic** in a **tree-like structure**. It helps in choosing between different options based on certain conditions or rules.

Purpose:

To **visually map out choices** and their possible outcomes.
Helps in **analyzing complex decisions** with multiple conditions.
Useful in **system analysis, programming, and AI** (especially in decision support systems and data mining).

Structure of a Decision Tree:

1. Root Node:

The first decision or condition in the tree.

2. Branches:

The **possible choices** or outcomes of a condition (e.g., Yes/No, True/False).

3. Leaf Nodes:

The **final outcomes or actions** taken after following a branch.

Example: Check if a student passes:

[Marks >= 40?]
/ \

```
        Yes       No
        /          \
[Print "Pass"]  [Print "Fail"]
```

Conclusion:

Decision trees are useful for making **logical decisions step-by-step**. They simplify complex conditions and are easy to read and understand.

3.9 Decision tables:

Definition:

A **decision table** is a **tabular method** used to represent and analyze **complex decision logic**. It lists all **possible conditions** and the corresponding **actions** to be taken based on those conditions.

Purpose:

To **systematically model** decision-making situations.
Helps in handling **multiple combinations** of conditions and outcomes.
Useful in **system analysis, design, and testing**.

Structure of a Decision Table:

Conditions	**Rule 1**	**Rule 2**	**Rule 3**	**Rule 4**
Age ≥ 18	Yes	Yes	No	No
ID Provided	Yes	No	Yes	No
Action	Issue ID	Reject	Reject	Reject

Conditions: Tests or criteria.
Rules: Different combinations of Yes/No or True/False.
Actions: What to do for each rule.

Conclusion:

Decision tables are excellent for **clarifying logic** and ensuring all possible **combinations of inputs** are considered. They reduce errors and improve decision transparency.